

Rocket Movies — Documentação Técnica do Sistema

1. Sumário Executivo & Visão Geral

O **Rocket Movies** é uma aplicação web full-stack desenvolvida para a gestão de um catálogo cinematográfico e sistema de avaliações, concebida sob os requisitos do projeto da Visagio - v(dev) para o Rocket Lab 2026.2. O sistema foi arquitetado com inspiração direta na experiência de utilizador (*UX*) e na identidade visual minimalista e imersiva do **Letterboxd**, apostando num tema escuro (*Dark Mode*), tipografia limpa, avaliações baseadas em meia-estrela e um fluxo de navegação fluido.

2. Arquitetura de Dados (Star Schema)

Para garantir alta performance, normalização adequada e facilidade de manutenção nas consultas analíticas e relacionais, a base de dados foi modelada seguindo o padrão de **Esquema Estrela (Star Schema)** adaptado para o ecossistema SQLite com SQLAlchemy (Async):

- **Tabela de Dimensão Principal (`dim_movies`):** Armazena os atributos centrais do filme (ID único, título, ano de lançamento, sinopse e URL do póster).
- **Tabelas de Dimensão de Apoio (`dim_genres` e `dim_people`):** Armazena os géneros cinematográficos e as pessoas envolvidas (separadas por tipo: Diretor ou Ator).
- **Tabelas de Ligação (*Bridge Tables*):** `bridge_movie_genre` e `bridge_movie_person` estabelecem as relações de muitos-para-muitos (N:M) entre os filmes e as suas respetivas dimensões.
- **Tabela de Transações/Avaliações (`movie_reviews`):** Registra as opiniões dos utilizadores, incluindo o nome do avaliador, a nota atribuída (com suporte a frações/meias-estrelas) e o comentário textual associado a um filme específico.

3. Stack Tecnológica & Decisões de Design

- **Backend (API REST):**
 - **FastAPI (Python):** Escolhido pela alta performance assíncrona, documentação interativa automática integrada (Swagger UI) e validação rigorosa de dados através do Pydantic.
 - **SQLAlchemy + SQLite:** Utilização de queries assíncronas (`AsyncSession`) para otimizar o fluxo de I/O, combinadas com o SQLite para garantir portabilidade imediata (dispensando a configuração complexa de servidores externos de base de dados).

- **Frontend (Single Page Application):**
 - **React & TypeScript:** Garantem tipagem estática segura, componentização modular e manutenibilidade a longo prazo.
 - **Vite & Tailwind CSS:** Proporcionam um ambiente de desenvolvimento ultrarrápido (*Hot Module Replacement*) e um design system utilitário altamente responsivo e coeso.
 - **React Router DOM:** Gestão de rotas dinâmicas e sincronização bidirecional de estados globais através de parâmetros de URL (`useSearchParams`).

4. Mapeamento de Funcionalidades Principais

- Catálogo Dinâmico e Paginação:**
 - Listagem paginada de filmes com cálculo dinâmico da média de estrelas baseada nas avaliações dos utilizadores.
- Sistema de Pesquisa Sincronizada com URL:**
 - A barra de pesquisa global presente no cabeçalho atualiza a URL em tempo real (`?search=`), permitindo partilhar estados de busca e garantindo total compatibilidade com o histórico de navegação do browser.
- Gestão Administrativa (CRUD Completo):**
 - **Criação (POST):** Registo de novos filmes com processamento automático de géneros, elenco e diretores.
 - **Edição (PUT):** Página dedicada que recupera os dados atuais do filme, limpando e atualizando as ligações relacionais de forma limpa.
 - **Exclusão (DELETE):** Remoção segura de registos diretamente pela interface de detalhes.
- Sistema de Avaliações (Reviews):**
 - Inserção de feedback interativo ordenado cronologicamente da mais recente para a mais antiga.
- Experiência de Utilizador (UX/UI de Transição):**
 - Uso de um *spinner* minimalista centralizado no topo com transição suave de opacidade (`transition-opacity`), mimetizando o comportamento de plataformas profissionais de streaming e crítica.

5. Estrutura de Diretórios

```

rocket-movies/
├── backend/
│   ├── app/
│   │   ├── api/v1/router.py    # Endpoints da API (Catálogo, CRUD, Reviews)
│   │   ├── db/base.py         # Configuração da sessão assíncrona do SQLAlchemy
│   │   ├── models.py          # Definição das tabelas ORM e tabelas Bridge
│   │   ├── schemas.py         # Contratos de validação Pydantic
│   │   └── main.py            # Ponto de entrada da aplicação FastAPI
│   └── requirements.txt       # Dependências Python

```

```
└─ frontend/
  └─ src/
    └─ components/Header.tsx # Cabeçalho global fixo com barra de pesquisa
    └─ pages/                # Páginas principais (Catalog, MovieDetails, NewMovie,
EditMovie)
    └─ App.tsx               # Configuração das rotas (React Router)
    └─ main.tsx              # Ponto de entrada React
  └─ package.json            # Dependências Node.js
```